

En el presente informe se encuentra la documentación de una de las partes más importantes del sistema para el login.

UsuarioDetailsService

Esta clase está alojada en la capa de servicios y se encarga de quién entra y qué puede hacer. Conecta el spring Security con la base de datos

Es un servicio de autenticación encargado de integrar la información de usuarios almacenada en la base de datos con el sistema de seguridad de Spring. Implementa la interfaz {UserService}, la cual es utilizada por Spring Security durante el proceso de login para cargar los datos del usuario.

Responsabilidades:

- Buscar el usuario en la base de datos a partir del nombre de usuario
- Obtener sus roles y permisos asociados
- Transformarlos en autoridades (GrantedAuthority)
- Retornar un objeto {UserDetails} que Spring pueda interpretar

Este servicio actúa como puente entre:

Base de datos (modelo relacional) y Spring Security (modelo de seguridad)

Descripción

Se declara que es un servicio de una clase pública llamada UsuarioDetailService que implementa la interfaz UserDetailsService mediante un contrato de métodos que utiliza spring para cargar usuarios en el proceso de la autenticación (login).

Se realiza una inyección de dependencias mediante el constructor que garantiza el llenado completo de los datos y que es obligatorio su implementación obteniendo los datos desde la base de datos mediante el repositorio usuarioRepository.

Se utiliza @Override que es la sobreescritura que usamos para decirle a spring que este método será el que el va entender cómo loadUserByUsername, que es propio de la interfaz UserDetailsService.

El método de tipo UserDetails llamado loadUserByUsername que tiene como parámetros el usuario que es único desde la base de datos. El método se encarga de buscar desde el repositorio al usuario o lanzar una excepción si no lo encuentra.

Se construye el objeto de usuario mediante un flujo de datos que va contener a los roles, los permisos de cada rol, se utiliza flatmap donde aplanar los datos (como sacarlos de su envoltorio para realizar una operación con ellos, en Java tienen que ser del mismo tipo de dato). Por cada rol hay una lista de autoridades (permisos).

Se realiza una lista auxiliar llamada autoridades que es de tipo SimpleGrantedAuthority que es la respuesta esperada de spring.

Agregamos los roles con el prefijo ROLE_ qué es lo que pide Spring para entender los roles y le agregamos el respectivo rol. Agregado a ello le insertamos por medio de un ForEach que recorre los permisos sin necesidad de índice y lo guarda en la lista, lo convierte nuevamente en un flujo de datos (Flatmap solo trabaja con flujos de datos) y lo convierte en la lista SimpleGrantedAuthority que va entender Spring.

Finalmente termina la comprensión de los datos a través de la construcción del objeto `user.builder` utilizando el usuario que viene del repositorio que se inyectó en el servicio, con la contraseña que proviene igualmente desde el repositorio, pero toma los roles y autoridades de una lista de tipo `SimpleGrantedAuthority` y finalmente declara la construcción del objeto y lo retorna

CÓDIGO

```
package com.example.demo.services;

import com.example.demo.models.Usuario;
import com.example.demo.repositories.UsuarioRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import org.springframework.security.core.authority.SimpleGrantedAuthority;
//Servicio encargado de gestionar quien entra y que puede hacer dentro del
sistema
@Service
public class UsuarioDetailsService implements UserDetailsService {

    /*Inyección por constructor spring detecta las dependencias y las inserta
    En este caso se dice que esta dependencia es obligatoria y no va cambiar
    conecta la base de datos específicamente el repositorio de usuario con
spring security*/
    private final UsuarioRepository usuarioRepository;
    @Autowired
    public UsuarioDetailsService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
    }
    /*
    Escriben en el login el usuario y esto lo busca y va al repositorio a
encontrarlo,
    si no lo encuentra lanza la excepción
    */
    @Override //sobreescritura personalizada del método de UserDetailsService
    public UserDetails loadUserByUsername(String usuario) throws
UsernameNotFoundException {
        Usuario entidadusuario = usuarioRepository.findByUsuario(usuario)
            .orElseThrow(() -> new UsernameNotFoundException("Usuario no
encontrado"));

        var authorities = entidadusuario.getRoles().stream() //flujo de datos
secuencial de usuarios para procesar uno a uno
            //Flatmap: aplanar todos los datos y los junta. Nombre del
rol y todos sus permisos
```

SGW - UsuarioDetailsService

```
        .flatMap(rol -> { //por cada rol hay una lista de las
autoridades(permisos)
            var autoridades = new
java.util.ArrayList<SimpleGrantedAuthority>(); //lista vacia para guardar el
rol y los permisos
            // agregacion del rol (con prefijo ROLE_) y todos sus
permisos para que spring lo entienda
            autoridades.add(new SimpleGrantedAuthority("ROLE_" +
rol.getNombre_rol()));
            rol.getPermisos().forEach(p -> autoridades.add(new
SimpleGrantedAuthority(p.getAccion())));
            return autoridades.stream(); //devuelve un flujo de datos
por que flatmap trabaja con streams
        })
        .toList(); //convierte ese stream en una lista de
SimpleGrantedAuthority para comprension de spring
        /* para cada rol del usuario, toma el rol y sus permisos, los
convierte en authorities,
        y junta todos en una sola lista*/

        return User.builder()
            .username(entidadusuario.getUsuario())
            .password(entidadusuario.getContraseña()) // contraseña
encriptada
            .authorities(authorities)
            .build();

        /*entonces termina la comprension de los datos a traves del
user.builder utilizando el usuario
        que viene del repositorio que se inyecto en el servicio, con la
contrasena pasa lo mismo, pero toma
        los roles y autoridades de una lista de tipo
        SimpleGrantedAuthority y finalmente declara la construccion del objeto
y lo retorna */
    }
}
```